

Materialized views

Knowledge Objectives

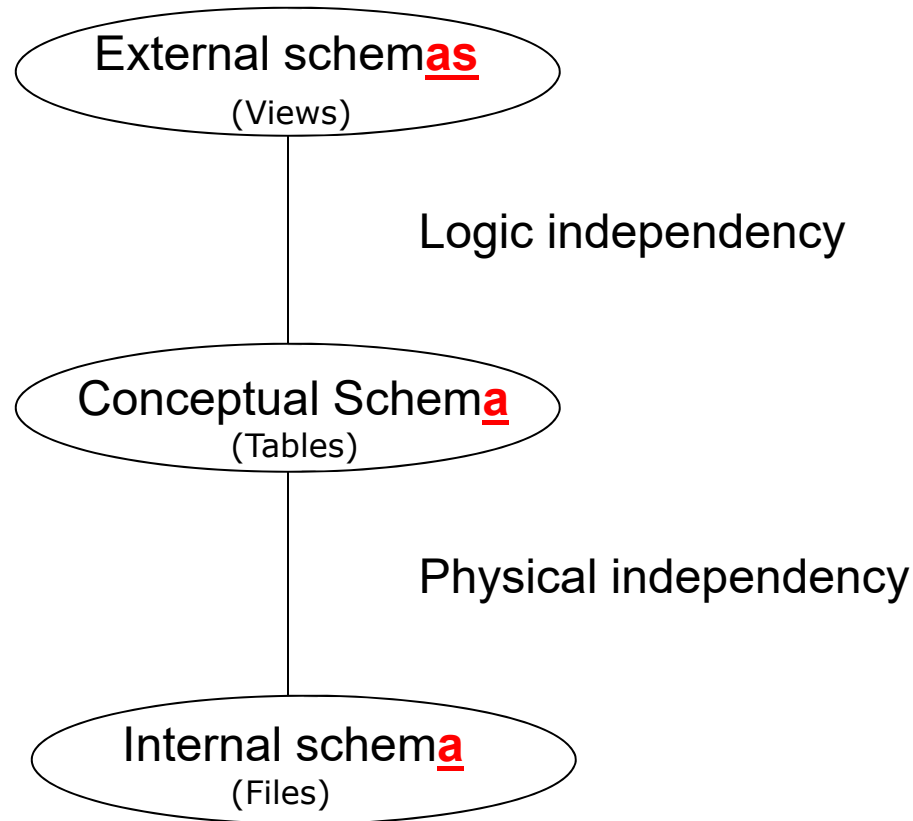
1. Explain the differences between the three levels in the ANSI/SPARC architecture, paying special attention to physical and logical independency
2. Explain the two differences between a view and a table
3. Explain the difference between a VIEW and a MATERIALIZED VIEW
4. Enumerate and distinguish the four problems associated to views
5. Discuss the benefits of a complete and an incremental view update
6. Enumerate when and how a materialized view can be refreshed

Application Objectives

1. Given a set of source tables (no more than 6) and some views over them (no more than 3), justify if a specific query over the tables can be rewritten in terms of the (materialized) views
2. Select a set of views to be when disk space is not limited, but the system is read-write
3. Given a materialized view and the statistics of the tables, decide whether it is worth or not an incremental update in front of a complete one

VIEW DEFINITION AND PROBLEMS

ANSI/SPARC architecture



Alternatives to implement a relation

- From the structures / data point of view
 - Tables
 - Data in disk (i.e., Materialized)
 - Non-materialized views
 - Definition in catalog (SQL statement)
 - Re-executed with every query
 - Materialized views
 - Data in disk (i.e., Materialized) and definition in catalog (SQL statement)
- From data retrieval point of view
 - Tables
 - Querying the materialized data
 - Non-materialized views
 - Transforming the query into another one over the underlying tables

$$V = F(R_1, R_2, \dots, R_n)$$
$$Q(V) \rightarrow Q'(R_1, R_2, \dots, R_n)$$

- Materialized views
 - Querying the pre-computed (i.e., Materialized) result of the query
 - Reduced to a synchronization problem

To improve query performance ...

- ... Build access structures
 - Index
 - Less attributes
 - Same number of entries as tuples
 - Less space than the table
 - Less I/O to be accessed
 - Useful for selectivity factors next to zero (very selective)
- ... Pre-calculate as much as possible
 - Redundant tables
 - Less attributes
 - Less tuples
 - Only those fulfilling the query predicate
 - Only one per combination of values of attributes in the GROUP BY
 - Less space than the table
 - Less I/O to be accessed

Problems associated to views

- ❑ Non materialized views
 - a) View expansion
 - ❑ Transform the query over the views into a query over the source tables
- ❑ Materialized views
 - b) Answering queries using views (i.e., query rewriting)
 - ❑ Transform an arbitrary query over the tables into a query over the available views
 - c) View updating
 - ❑ Changes in the sources are, potentially, propagated to the view
- ❑ Both
 - d) Update through views
 - ❑ Propagate the changes in the view to the sources by means of a translation process

Materialized views in Oracle

```
CREATE MATERIALIZED VIEW <name>
```

```
[BUILD {IMMEDIATE|DEFERRED}]
```

```
[REFRESH
```

```
[{NEVER|FAST|COMPLETE|FORCE}]
```

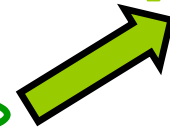
```
[ON DEMAND|ON COMMIT|NEXT <date> }]]
```

```
[FOR UPDATE]
```

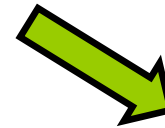
```
[{DISABLE|ENABLE} QUERY REWRITE]
```

```
AS <query>;
```

View updating



Answering queries using views



ANSWERING QUERIES USING VIEWS

Answering queries using views

- ❑ The problem of deciding whether it is possible to rewrite a query in terms of existing materialized views or not is computationally complex
 - DBMSs restrict the search space to common cases by using rules
- ❑ Principles:
 - a) Query predicate must be subsumed by that of the view
 - b) Aggregation level in the query must be higher or equal to that in the materialized view
 - Functional dependencies can be used to check it
 - c) Aggregates must coincide (or be computable)

Predicate subsumption

Table

A	B	C	D
1			
2			
3			
4			
5			

A IN (2,5)

Query 1

A	B	C	D
2			
5			

Odd numbers

MView

A	B	C	D
1			
3			
5			

A IN (3,5)

Query 2

A	B	C	D
3			
5			

Aggregation level

Table

<u>A</u>	<u>B</u>	<u>C</u>	D
1	a	x	
1	a	y	
1	b	x	
2	a	x	
2	a	y	

GROUP BY C

Query 1

<u>C</u>	f(D)
x	
y	

GROUP BY A,B

MView

<u>A</u>	<u>B</u>	f(D)
1	a	
1	b	
2	a	

GROUP BY A

Query 2

<u>A</u>	f(D)
1	
2	

Aggregation function

Table

<u>A</u>	<u>B</u>	<u>C</u>	D
1	a	x	
1	a	y	
1	b	x	
2	a	x	
2	a	y	



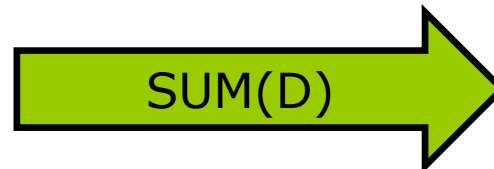
Query 1

<u>A</u>	COUNT(*)
1	
2	

SUM

MView

<u>A</u>	<u>B</u>	SUM(D)
1	a	
1	b	
2	a	

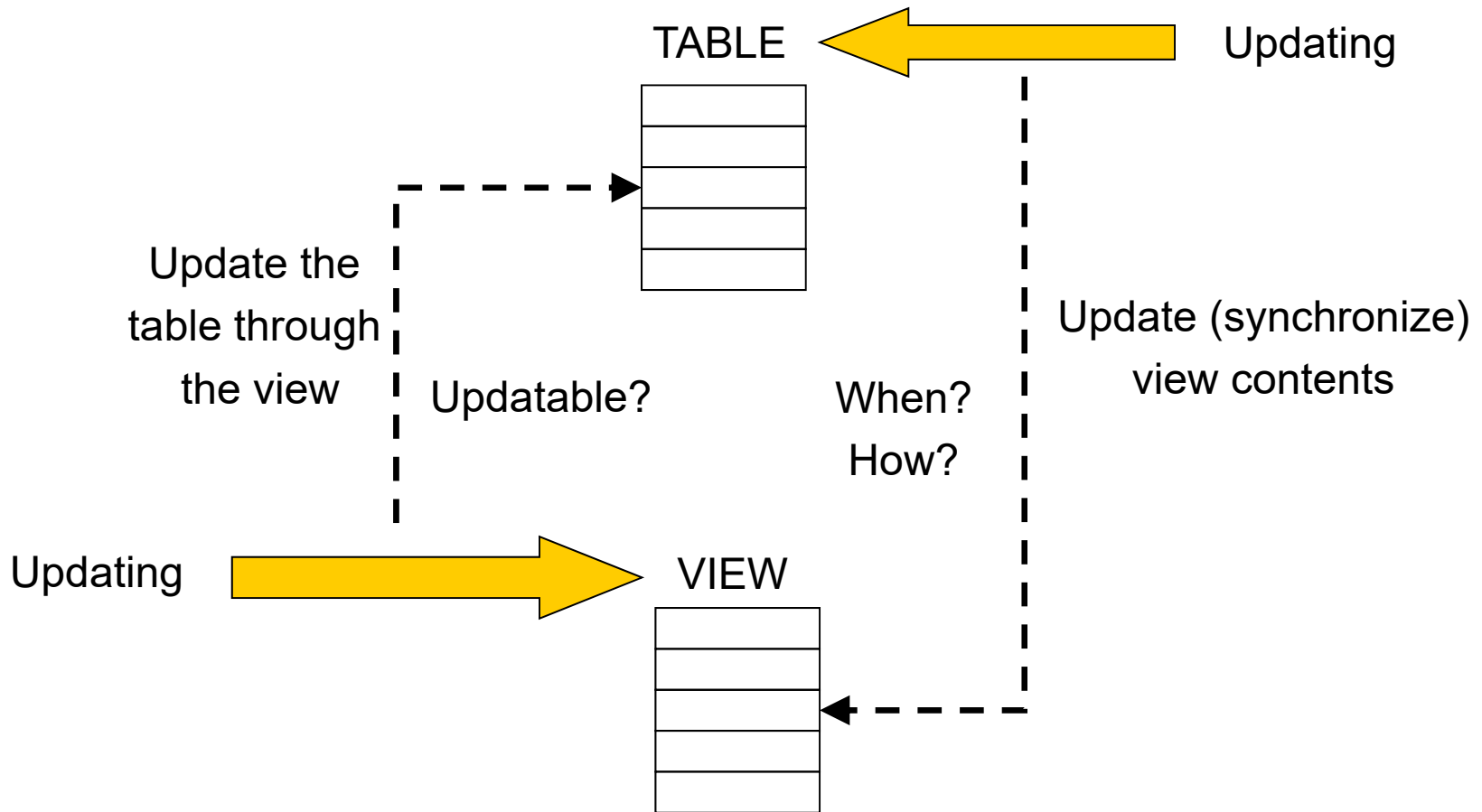


Query 2

<u>A</u>	SUM(D)
1	
2	

VIEW UPDATING

Views and modifications



View updating (how)

- ❑ Complete update
 - All instances are regenerated
 - ❑ Clearly inefficient?
 - ❑ Always possible
- ❑ Incremental update (called “fast” in Oracle)
 - Only instances that changed are regenerated
 - ❑ Much more efficient?
 - ❑ Not always possible
 - Depends on the information available

Fast materialized views in Oracle

- ❑ On commit refresh is only possible for views allowing incremental (fast) updates
- ❑ A log must be defined for every source table
 - Only one log per table is allowed!
 - Stores rows describing changes from last refresh
 - Tuples should be univocally identified (ROWID or PK needed)

```
CREATE MATERIALIZED VIEW LOG ON table  
[WITH clause] [NEW values]
```

- ❑ Both the log and the view definition (query) must fulfill a set of constraints
 - Basic queries (without groupings nor joins)
 - Join queries
 - Grouping queries
- ❑ Oracle explanation for *fast* update
 - BEGIN DBMS_MVIEW.EXPLAIN_MVIEW('materialized_view_name'); END;

Oracle DataWarehousing Guide, chapter 9 “Basic Materialized Views”

MATERIALIZED VIEW SELECTION

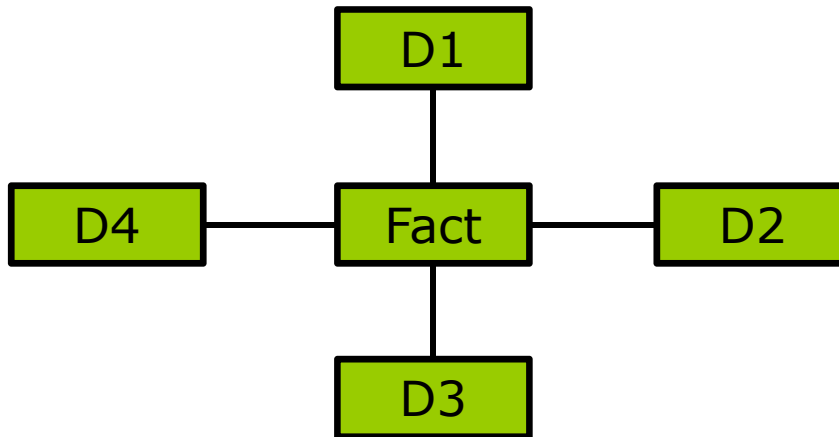
Problems in pre-computing

- Cost
 - Space
 - Time
 - Query vs Modification frequency
- Consistency and rewriting control
 - Using materialized views
 - Alternatively implemented with triggers

Combinatorial aggregation explosion

- Choosing the best combination of views to be materialized is NP-complex
 - A fact table with D dimension tables with L aggregation levels (excluding the “All” level) for each one, would generate $(L+1)^D$ possible materialized views

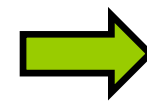
Example of combinatorial explosion



GROUP BY D1, D2, D3, D4;

·
·
·

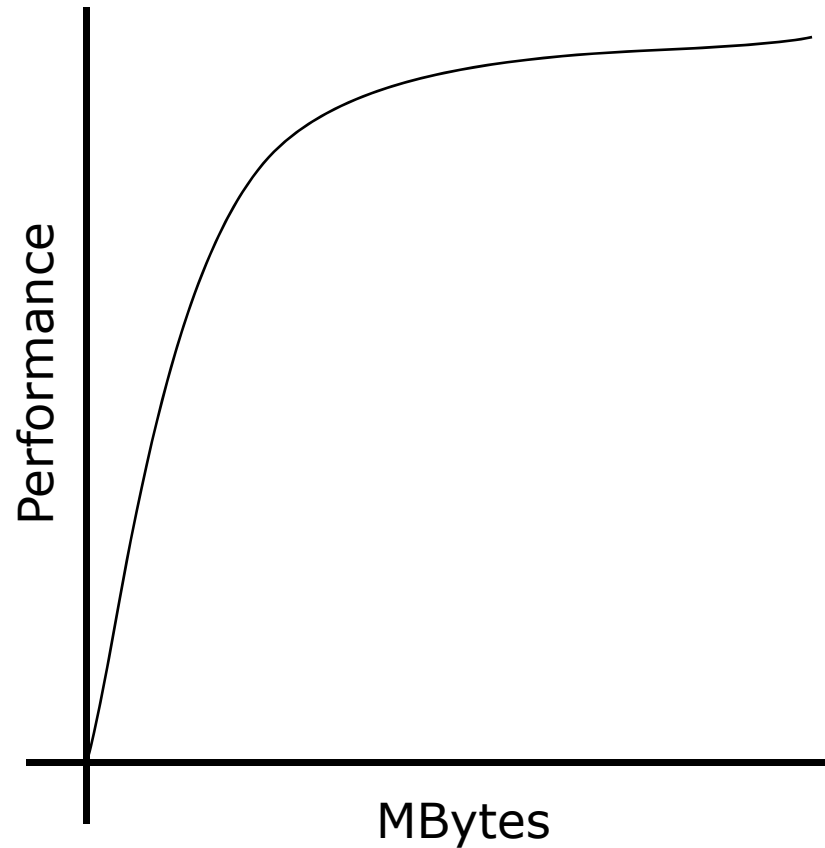
GROUP BY ();



$(L+1)^D$

D1	D2	D3	D4
1	1	1	1
1	1	1	0
1	1	0	1
1	1	0	0
1	0	1	1
1	0	1	0
1	0	0	1
1	0	0	0
0	1	1	1
0	1	1	0
0	1	0	1
0	1	0	0
0	0	1	1
0	0	1	0
0	0	0	1
0	0	0	0

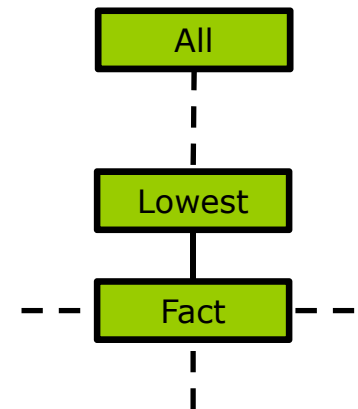
Materialization trade-off



Is our Update Window enough?

Solutions to the aggregation explosion

- ❑ The sparser the basic table/cube, (proportionally) the more space aggregates will use
 - Twelve days per year may generate twelve months per year
- ❑ Heuristics:
 - ❑ Materialize lower aggregation levels
 - ❑ They slightly reduce the size and solve many queries
 - ❑ Materialize higher aggregation levels
 - ❑ They are queried very often
 - ❑ Materialize a view if it solves a critical query or many queries
 - ❑ Do not materialize a view if it is a close ancestor in the aggregation lattice of an already materialized view
 - ❑ Each tuple comes from the aggregation of at least 10
- ❑ Modify the set of mviews as user needs evolve



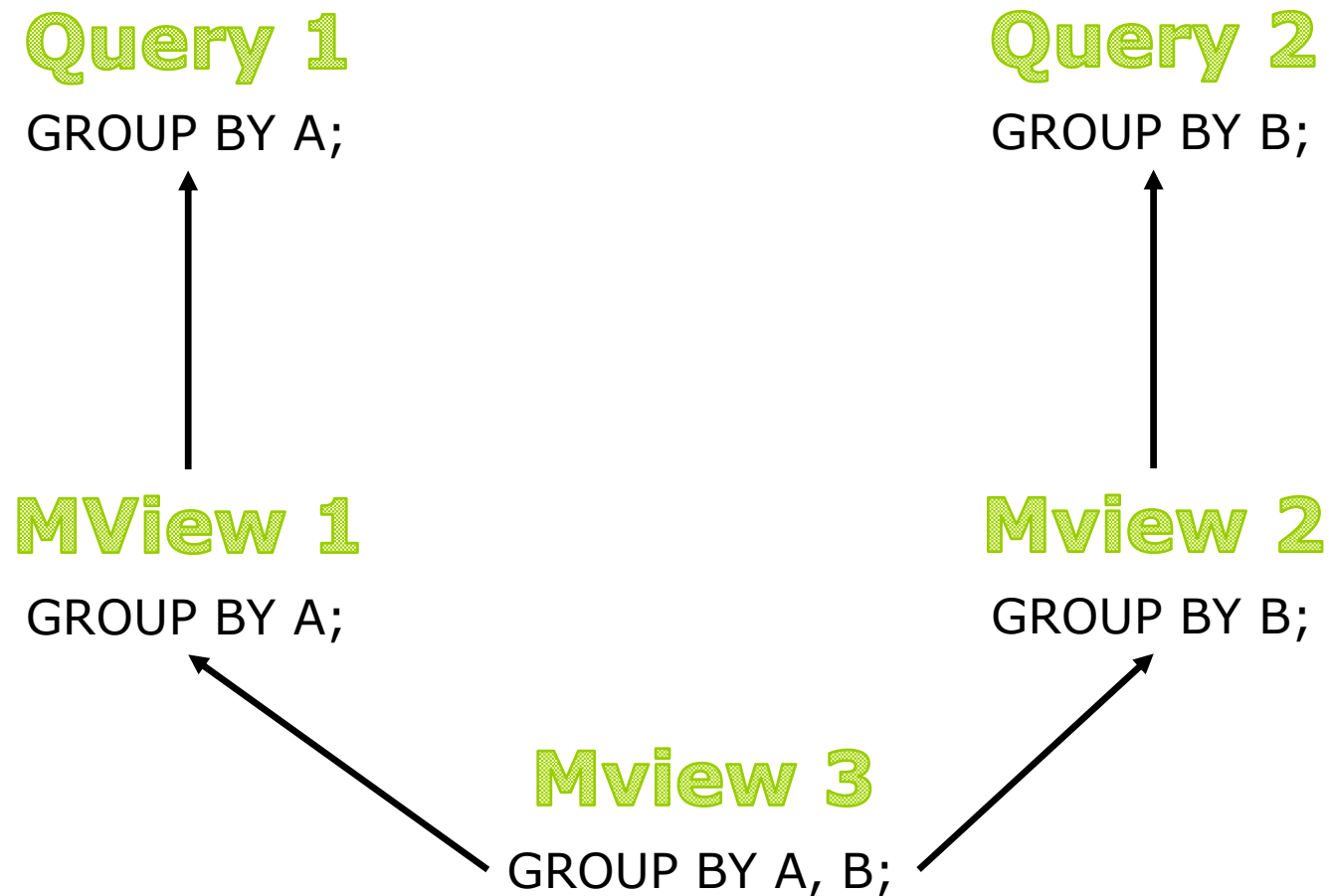
Candidate views to be materialized

Given a workload $W = \{q_1, q_2, q_3, \dots\}$, and identifying queries by their GROUP BY clause, candidate views v_i are those that:

a) $GB(v_i) = GB(q_j)$

b) $GB(v_i) = \bigcup_{q_j \in Q} GB(q_j)$, where $Q \subseteq W$

Example of candidate materialized views



CLOSING

Summary

- ❑ ANSI/SPARC architecture
- ❑ Problems when dealing with views
 - [View expansion]
 - Answering queries using views
 - View updating
 - [Update through views]
- ❑ Pre-aggregation
- ❑ Materialized view selection

Bibliography

- George Gardarin and Patrick Valduriez. Relational databases and knowledge bases. Addison Wesley Publishing Company, 1989
- T. Teorey et al. *Database modeling and design*. Morgan Kaufmann Publishers, 2006. 4th edition
- M. Golfarelli and S. Rizzi. *Data Warehouse Design*. McGrau-Hill, 2009